```
# Step 1: Import Libraries
import numpy as np
import matplotlib.pyplot as plt
import time
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Input, Flatten
from tensorflow.keras.utils import to_categorical

# Step 2: Load Dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# Step 3: Preprocessing
X_train = X_train.astype("float32") / 255.0
X_test = X_test.astype("float32") / 255.0
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)
# Step 4: Create Neural Network Model
def create_model(activation):
    model = Sequential([
        Input(shape=(28, 28)),
        Flatten(),
        Dense(128, activation=activation),
        Dense(64, activation=activation),
        Dense(10, activation="softmax")
    ])
    model.compile(
        optimizer="adam",
        loss="categorical_crossentropy",
        metrics=["accuracy"]
    )
    return model
# Step 5: Train Models with Different Activations
activations = ["sigmoid", "tanh", "relu"]
results = {}
histories = {}
for act in activations:
    print(f"\nTraining with {act} activation")
    model = create_model(act)
    start = time.time()
    history = model.fit(
        X_train, y_train,
        epochs=5,
        batch_size=128,
        validation_split=0.2,
        verbose=1
    )
    training_time = time.time() - start
    loss, accuracy = model.evaluate(
        X_test, y_test, verbose=0
    )
    results[act] = {
        "accuracy": accuracy,
        "loss": loss,
        "time": training_time
    }
    histories[act] = history.history
# Step 6: Compare Test Accuracy and Training Time
print("\nFinal Comparison")
print("-" * 50)
print(f"{'Activation':<12}{'Accuracy':<15}{'Time (s)':<12}")
for act in activations:
    print(
        f"{act:<12}"
        f"{results[act]['accuracy']:.4f}"
        f"{'':<10}"
        f"{results[act]['time']:.2f}"   )
# Step 7: Plot Training and Validation Accuracy
plt.figure(figsize=(10, 5))
for act in activations:
    plt.plot(
        histories[act]["accuracy"],
        label=act + " Training"
    )
```

```
    plt.plot(
        histories[act]["val_accuracy"],
        linestyle="--",
        label=act + " Validation"
    )
plt.title("Activation Function Comparison on
MNIST")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
# Step 8: Plot Test Accuracy Comparison
plt.figure(figsize=(7, 4))
accuracy_values = [
    results[act]["accuracy"] * 100 for act in
activations
]
plt.bar(activations, accuracy_values)
plt.title("Test Accuracy for Different Activation
Functions")
plt.xlabel("Activation Function")
plt.ylabel("Test Accuracy (%)")
plt.tight_layout()
plt.show()
```

```
Final Comparison
----------------------------------------
Activation  Accuracy        Time (s)
sigmoid     0.9555           6.16
tanh        0.9720           5.47
relu        0.9727           5.39
```

(graph in exp8 output)

